

COURS COMPLET

C# + React

De zero a un projet full-stack

- 1 C# — Les bases du langage
- 2 ASP.NET Core — Web API
- 3 Entity Framework Core
- 4 React — Composants et Hooks
- 5 REST API — Concepts clés
- 6 Git — Versionner son code
- 7 Docker — Containeriser l'app

Niveau : Débutant

Ce cours couvre tous les concepts nécessaires pour construire le projet Todo App de A à Z. Chaque module est indépendant : tu peux le lire dans l'ordre ou revenir consulter un concept spécifique en cas de blocage.

Module	Contenu
1 — C# Les bases	Types, classes, enum, interfaces, async/await, LINQ
2 — ASP.NET Core	Controllers, routing, DI, codes HTTP, middleware
3 — Entity Framework	ORM, DbContext, migrations, CRUD avec EF
4 — React	Composants, JSX, useState, useEffect, props, Axios
5 — REST API	HTTP, verbes, JSON, CORS
6 — Git	Commits, branches, GitHub, Conventional Commits
7 — Docker	Images, containers, Dockerfile, docker-compose



C# — Les bases du langage

Types · Classes · Interfaces · async/await · LINQ

1.1 Types de base

C# est un langage **fortement type** : chaque variable a un type fixe connu a la compilation. Cela evite de nombreuses erreurs au runtime.

Type	Valeur exemple	Description
int	42	Entier 32 bits
long	9_000_000_000L	Entier 64 bits
double	3.14	Virgule flottante 64 bits
decimal	99.99m	Virgule fixe (finance, prix)
bool	true / false	Booleen
string	"bonjour"	Chaine de caracteres UTF-16
char	'a'	Caractere unique
DateTime	DateTime.UtcNow	Date et heure
int?	null	Nullable — peut valoir null

```
// Variables
int age = 25;
string prenom = "Alice";
bool estActif = true;

// var = le compilateur deduit le type (meme chose, juste plus court)
var age = 25; // C# sait que c'est un int
var prenom = "Alice"; // C# sait que c'est un string

// Nullable : string? peut etre null, string ne peut pas
string? description = null; // OK
string titre = null; // ERREUR de compilation en C# moderne

// Interpolation de string
Console.WriteLine($"Bonjour {prenom}, tu as {age} ans");
```

1.2 Classes et proprietes

Une classe est un **modele** (blueprint) pour creer des objets. Elle regroupe des donnees (proprietes) et des comportements (methodes).

```
public class TodoTask
{
    // Propriete auto-implementee (get/set generes automatiquement)
    public int Id { get; set; }
```

```
// Valeur par default : string.Empty = ""
public string Title { get; set; } = string.Empty;

// ? = peut etre null (reference nullable)
public string? Description { get; set; }

// Default via expression
public bool IsCompleted { get; set; } = false;
public DateTime CreatedAt { get; set; } = DateTime.UtcNow;

// Methode
public void Complete()
{
    IsCompleted = true;
}

// Creer un objet (instancier la classe)
var task = new TodoTask();
task.Title = "Apprendre C#";
task.Complete();

// Syntaxe objet initializer (plus concise)
var task2 = new TodoTask { Title = "Apprendre React", Id = 2 };
```

1.3 Enum

Un enum definit un ensemble de valeurs nommees. Utilise un enum plutot qu'une string pour représenter un choix fixe : cela evite les fautes de frappe et offre l'IntelliSense.

```
public enum Priority
{
    Low = 0,
    Medium = 1,
    High = 2
}

// Utilisation
var p = Priority.High;

// Convertir enum <-> string
string s = p.ToString(); // "High"
Priority parsed = Enum.Parse<Priority>("Low"); // Priority.Low

// Dans une condition
if (p == Priority.High)
    Console.WriteLine("Urgent !");
```

1.4 Interfaces

Une interface est un **contrat** : elle liste les methodes qu'une classe *doit* implementer, sans donner l'implementation. C'est la base du principe d'**inversion de dependance** (SOLID).

```
// 1. Definir l'interface (le contrat)
public interface ITaskService
{
    Task<IEnumerable<TaskDto>> GetAllAsync();
    Task<TaskDto?> GetByIdAsync(int id);
    Task<TaskDto> CreateAsync(CreateTaskDto dto);
    Task<bool> DeleteAsync(int id);
}

// 2. L'implementation respecte le contrat
public class TaskService : ITaskService
{
    public async Task<IEnumerable<TaskDto>> GetAllAsync()
    {
        // implementation reelle avec la BDD
    }
    // ... autres methodes
}

// 3. Le code consommateur connait seulement l'interface
ITaskService service = new TaskService(...);
var tasks = await service.GetAllAsync();

// => En test, on peut injecter un MockTaskService sans toucher au reste
```

Pourquoi utiliser des interfaces ?

- Facilite les tests unitaires (injecter un faux service) - Permet de changer l'implementation sans modifier le code appelant - Exprime clairement le "contrat" entre les couches de l'application

1.5 async / await

Les operations longues (acces BDD, appels reseau) ne doivent pas bloquer le thread. **async/await** permet d'attendre sans bloquer : le thread est libre pour traiter d'autres requetes pendant l'attente.

```
// SANS async : bloque le thread jusqu'a la reponse SQL
public List<TodoTask> GetAll()
{
    return _context.Tasks.ToList(); // bloquant
}

// AVEC async : libere le thread pendant l'accès BDD
public async Task<List<TodoTask>> GetAllAsync()
{
    return await _context.Tasks.ToListAsync(); // non-bloquant
}

// Regles :
// - Une methode async retourne Task (void), Task<T> (valeur) ou ValueTask<T>
// - On met await devant toute expression async
// - async se propage : si tu awaites, ta methode doit etre async
// - Ne jamais utiliser .Result ou .Wait() -> risque de deadlock
```

```
// Exemple complet
public async Task<TaskDto?> GetByIdAsync(int id)
{
    var task = await _context.Tasks.FindAsync(id); // await la BDD
    if (task is null) return null; // pattern null check moderne
    return MapToDto(task); // methode synchrone
}
```

1.6 LINQ — requetes sur les collections

LINQ (Language Integrated Query) permet de filtrer, trier et transformer des collections avec une syntaxe fluide et lisible, similaire au SQL.

```
var tasks = new List<TodoTask> { ... };

// Filtrer
var actives = tasks.Where(t => !t.IsCompleted).ToList();

// Trier
var sorted = tasks.OrderByDescending(t => t.CreatedAt).ToList();

// Transformer (projection)
var titles = tasks.Select(t => t.Title).ToList();

// Chainer les operations
var result = tasks
    .Where(t => t.Priority == Priority.High && !t.IsCompleted)
    .OrderBy(t => t.DueDate)
    .Select(t => new { t.Title, t.DueDate })
    .ToList();

// Aggregation
int total = tasks.Count();
bool anyUrgent = tasks.Any(t => t.Priority == Priority.High);
var firstUrgent = tasks.FirstOrDefault(t => t.Priority == Priority.High);

// AVEC Entity Framework : les memes methodes generent du SQL !
var dbResult = await _context.Tasks
    .Where(t => !t.IsCompleted)
    .OrderByDescending(t => t.CreatedAt)
    .ToListAsync(); // SELECT * FROM Tasks WHERE IsCompleted=0 ORDER BY CreatedAt DESC
```

2.1 Architecture d'une requete HTTP

Quand une requete arrive, elle traverse un **pipeline de middleware** avant d'atteindre le controller. Chaque middleware peut modifier la requete ou la reponse, ou court-circuiter le pipeline.

```
// Program.cs – configuration du pipeline
var builder = WebApplication.CreateBuilder(args);

// 1. Enregistrer les services (avant Build)
builder.Services.AddControllers();
builder.Services.AddDbContext<AppDbContext>(...);
builder.Services.AddScoped<ITaskService, TaskService>();

var app = builder.Build();

// 2. Configurer le pipeline (apres Build) – ordre important !
app.UseHttpsRedirection(); // Middleware 1
app.UseCors("AllowReact"); // Middleware 2
app.UseAuthentication(); // Middleware 3 (si auth)
app.UseAuthorization(); // Middleware 4 (si auth)
app.MapControllers(); // Route vers les controllers
app.Run();
```

2.2 Controllers et routing

```
// [ApiController] active : validation auto des DTOs, inference [FromBody]/[FromRoute]
// [Route("api/[controller]")] -> /api/tasks (le nom de la classe sans "Controller")
[ApiController]
[Route("api/[controller]")]
public class TasksController : ControllerBase
{
    private readonly ITaskService _taskService;

    // Injection de dependance : ASP.NET fournit ITaskService automatiquement
    public TasksController(ITaskService taskService)
    {
        _taskService = taskService;
    }

    [HttpGet] // GET /api/tasks
    public async Task<IActionResult> GetAll()
    => Ok(await _taskService.GetAllAsync());

    [HttpGet("{id}")] // GET /api/tasks/5
    public async Task<IActionResult> GetById(int id)
    {
        var task = await _taskService.GetByIdAsync(id);
```

```

return task is null ? NotFound() : Ok(task);
}

[HttpPost] // POST /api/tasks
public async Task<IActionResult> Create([FromBody] CreateTaskDto dto)
{
    var created = await _taskService.CreateAsync(dto);
    return CreatedAtAction(nameof(GetById), new { id = created.Id }, created);
}

[HttpPut("{id}")] // PUT /api/tasks/5
public async Task<IActionResult> Update(int id, [FromBody] UpdateTaskDto dto)
{
    var updated = await _taskService.UpdateAsync(id, dto);
    return updated is null ? NotFound() : Ok(updated);
}

[HttpDelete("{id}")] // DELETE /api/tasks/5
public async Task<IActionResult> Delete(int id)
{
    var deleted = await _taskService.DeleteAsync(id);
    return deleted ? NoContent() : NotFound();
}
}

```

2.3 Injection de dependance (DI)

La DI est un patron de conception : plutot que de creer les objets manuellement (new TaskService()), on enregistre les services dans un conteneur et ASP.NET les cree et les passe automatiquement aux constructeurs.

```

// Enregistrement dans Program.cs
builder.Services.AddScoped<ITaskService, TaskService>();
// ^ ^interface ^implementation
// |
// Duree de vie :
// AddScoped -> une instance par requete HTTP (le plus courant)
// AddSingleton -> une seule instance pour toute la duree de l'app
// AddTransient -> une nouvelle instance a chaque injection

// Injection dans le constructeur (fonctionne partout)
public class TasksController : ControllerBase
{
    private readonly ITaskService _taskService;

    public TasksController(ITaskService taskService) // <- inject ici
    {
        _taskService = taskService;
    }
}

```

AddScoped vs AddSingleton vs AddTransient

Scoped (recommande pour les services avec BDD) : une instance par requete HTTP. Si deux classes dans la meme requete demandent ITaskService, elles obtiennent la meme instance. Singleton : meme instance partout, toujours. Utilise pour les caches, la config. Transient : nouvelle instance a chaque injection. Utilise pour les objets legers sans etat.

2.4 Codes HTTP — reference

Code	Nom	Quand l'utiliser	Methode C#
200	OK	Succes avec donnees en reponse (GET, PUT)	Ok(data)
201	Created	Ressource cree (POST)	CreatedAtAction(...)
204	No Content	Succes sans corps (DELETE)	NoContent()
400	Bad Request	Donnees invalides envoyees	BadRequest(...)
401	Unauthorized	Non authentifie	Unauthorized()
403	Forbidden	Authentifie mais pas autorise	Forbid()
404	Not Found	Ressource introuvable	NotFound()
500	Server Error	Erreur inattendue cote serveur	(exception)

3.1 Qu'est-ce qu'un ORM ?

Entity Framework Core est un **ORM** (Object-Relational Mapper). Il fait le pont entre les objets C# et les tables SQL. Tu manipules des objets C# normaux — EF traduit en SQL automatiquement.

C# (EF Core)	SQL equivalent
<code>_context.Tasks.ToListAsync()</code>	<code>SELECT * FROM Tasks</code>
<code>_context.Tasks.FindAsync(5)</code>	<code>SELECT * FROM Tasks WHERE Id = 5</code>
<code>_context.Tasks.Add(task)</code>	<code>INSERT INTO Tasks VALUES (...)</code>
<code>await _context.SaveChangesAsync()</code>	<code>COMMIT</code>
<code>_context.Tasks.Remove(task)</code>	<code>DELETE FROM Tasks WHERE Id = ...</code>
<code>_context.Tasks.Where(t => !t.IsCompleted)</code>	<code>SELECT * FROM Tasks WHERE IsCompleted = 0</code>

3.2 AppDbContext

```
using Microsoft.EntityFrameworkCore;

public class AppDbContext : DbContext
{
    // Constructeur : recoit la config (connection string, provider)
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }

    // Chaque DbSet<T> = une table dans la BDD
    public DbSet<TodoTask> Tasks { get; set; }

    // Configuration avancee et donnees de seed
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        // Contrainte : Title ne peut pas etre vide
        modelBuilder.Entity<TodoTask>()
            .Property(t => t.Title)
            .IsRequired()
            .HasMaxLength(200);

        // Donnees de depart (seed data)
        modelBuilder.Entity<TodoTask>().HasData(
            new TodoTask { Id = 1, Title = "Exemple", IsCompleted = false,
                Priority = Priority.Medium, CreatedAt = DateTime.UtcNow }
        );
    }
}
```

```
// Enregistrement dans Program.cs
builder.Services.AddDbContext<AppDbContext>(options =>
options.UseSqlite("Data Source=todo.db"));
```

3.3 Migrations

Les migrations versionnent le schema de ta base de donnees. A chaque modification du Model, tu crees une migration : EF genere le code SQL de la difference.

```
# Installer l'outil EF (une seule fois)
dotnet tool install --global dotnet-ef

# Creer une migration (apres avoir modifie un Model)
dotnet ef migrations add InitialCreate
# -> Cree un fichier Migrations/20240101_InitialCreate.cs

# Appliquer la migration (cree/modifie la BDD)
dotnet ef database update

# Voir les migrations appliquees
dotnet ef migrations list

# Annuler la derniere migration
dotnet ef migrations remove

# En code : appliquer automatiquement au demarrage
using (var scope = app.Services.CreateScope())
{
    var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
    db.Database.Migrate(); // Cree la BDD si elle n'existe pas
}
```

3.4 Operations CRUD dans le Service

```
public class TaskService : ITaskService
{
    private readonly AppDbContext _context;
    public TaskService(AppDbContext context) { _context = context; }

    // CREATE
    public async Task<TaskDto> CreateAsync(CreateTaskDto dto)
    {
        var task = new TodoTask
        {
            Title = dto.Title,
            Description = dto.Description,
            Priority = Enum.Parse<Priority>(dto.Priority, ignoreCase: true),
            CreatedAt = DateTime.UtcNow
        };
        _context.Tasks.Add(task);
        await _context.SaveChangesAsync();
        return MapToDto(task);
    }
}
```

```

}

// READ ALL
public async Task<IEnumerable<TaskDto>> GetAllAsync()
=> (await _context.Tasks.OrderByDescending(t => t.CreatedAt).ToListAsync())
.Select(MapToDto);

// UPDATE (patch partiel)
public async Task<TaskDto?> UpdateAsync(int id, UpdateTaskDto dto)
{
var task = await _context.Tasks.FindAsync(id);
if (task is null) return null;

// On ne met a jour que les champs non-null
if (dto.Title is not null) task.Title = dto.Title;
if (dto.IsCompleted is not null) task.IsCompleted = dto.IsCompleted.Value;
if (dto.Priority is not null) task.Priority = Enum.Parse<Priority>(dto.Priority, true);

await _context.SaveChangesAsync();
return MapToDto(task);
}

// DELETE
public async Task<bool> DeleteAsync(int id)
{
var task = await _context.Tasks.FindAsync(id);
if (task is null) return false;
_context.Tasks.Remove(task);
await _context.SaveChangesAsync();
return true;
}

// Conversion prive Model -> DTO
private static TaskDto MapToDto(TodoTask t) => new()
{
Id = t.Id, Title = t.Title, Description = t.Description,
IsCompleted = t.IsCompleted, Priority = t.Priority.ToString(),
CreatedAt = t.CreatedAt, DueDate = t.DueDate
};
}

```

4.1 Qu'est-ce que React ?

React est une bibliothèque JavaScript pour construire des interfaces utilisateur. Le principe cle : l'UI est une **fonction de l'état**. Quand l'état change, React recalculé et met à jour uniquement ce qui a changé (Virtual DOM).

```
// Un composant React = une fonction qui retourne du JSX
// JSX = mélange HTML + JavaScript (transpile en React.createElement(...))

function Greeting({ name }) {
  return (
    <div className="greeting"> {/* className, pas class ! */}
    <h1>Bonjour, {name} !</h1> {/* {} = expression JS dans JSX */}
    <p>Bienvenue sur l'app.</p>
    </div>
  )
}

// Utilisation dans un autre composant
function App() {
  return <Greeting name="Alice" />
}
```

4.2 useState — gerer l'état local

useState est le hook fondamental. Il permet à un composant de "se souvenir" d'une valeur entre les rendus. Modifier l'état via la fonction setter déclenche automatiquement un re-rendu.

```
import { useState } from "react"

function Counter() {
  // useState(valeurInitiale) retourne [valeurActuelle, fonctionSetter]
  const [count, setCount] = useState(0)
  const [name, setName] = useState("")

  return (
    <div>
      <p>Compteur : {count}</p>
      <button onClick={() => setCount(count + 1)}>+1</button>
      <button onClick={() => setCount(prev => prev - 1)}>-1</button>
      {/* Forme avec callback : preferee quand le nouvel etat depend de l'ancien */}

      <input
        value={name}
        onChange={(e) => setName(e.target.value)}
        placeholder="Ton prenom"
      />
      <p>Bonjour {name} !</p>
    </div>
  )
}
```

```

</div>
)
}

// Regles importantes :
// 1. Jamais modifier l'etat directement : tasks.push(x) <- INTERDIT
// Toujours via le setter : setTasks([...tasks, x]) <- CORRECT
// 2. Les objets et tableaux sont immuables :
// setTask({ ...task, isCompleted: true }) <- copie + modification

```

4.3 useEffect — effets de bord

useEffect execute du code apres le rendu. C'est ici qu'on fait les appels API, les abonnements a des evenements, les timers, etc. — tout ce qui est "en dehors" du rendu pur.

```

import { useState, useEffect } from "react"
import taskService from "../services/taskService"

function App() {
  const [tasks, setTasks] = useState([])
  const [loading, setLoading] = useState(true)

  // useEffect(fonction, [dependances])
  // [] vide = s'execute UNE SEULE FOIS apres le premier rendu (= componentDidMount)
  useEffect(() => {
    const load = async () => {
      try {
        const data = await taskService.getAll()
        setTasks(data)
      } finally {
        setLoading(false)
      }
    }
    load()
  }, []) // <- tableau de dependances vide

  // [userId] = s'execute a chaque fois que userId change
  useEffect(() => {
    fetchUserData(userId)
  }, [userId])

  // Aucun tableau = s'execute apres CHAQUE rendu (rarement utile)
  useEffect(() => {
    document.title = `${tasks.length} taches`
  })

  if (loading) return <p>Chargement...</p>
  return <TaskList tasks={tasks} />
}

```

4.4 Props — communication parent -> enfant

Les props sont des parametres passes d'un composant parent a un enfant. Elles sont **en lecture seule** : l'enfant ne peut pas les modifier directement. Pour remonter une information vers le parent, on passe une **fonction callback** en prop.

```
// Composant enfant : recoit les donnees et les callbacks en props
function TaskCard({ task, onToggle, onDelete }) {
  return (
    <div className={task.isCompleted ? "done" : ""}>
      <h3>{task.title}</h3>
      <button onClick={() => onToggle(task)}>
        {task.isCompleted ? "Reactivier" : "Terminer"}
      </button>
      <button onClick={() => onDelete(task.id)}>Supprimer</button>
    </div>
  )
}

// Composant parent : possede l'etat et passe les handlers
function App() {
  const [tasks, setTasks] = useState([])

  const handleToggle = async (task) => {
    const updated = await taskService.update(task.id, { isCompleted: !task.isCompleted })
    setTasks(prev => prev.map(t => t.id === updated.id ? updated : t))
  }

  const handleDelete = async (id) => {
    await taskService.delete(id)
    setTasks(prev => prev.filter(t => t.id !== id))
  }

  return (
    <div>
      {tasks.map(task => (
        <TaskCard
          key={task.id} // obligatoire pour les listes
          task={task}
          onToggle={handleToggle}
          onDelete={handleDelete}
        />
      ))}
    </div>
  )
}
```

4.5 Appels HTTP avec Axios

```
// installation : npm install axios
import axios from "axios"

// Service centralise (src/services/taskService.js)
const BASE = "/api/tasks"

export default {
```

```

getAll: () => axios.get(BASE).then(r => r.data),
getById: (id) => axios.get(`${BASE}/${id}`).then(r => r.data),
create: (data) => axios.post(BASE, data).then(r => r.data),
update: (id, data) => axios.put(`${BASE}/${id}`, data).then(r => r.data),
delete: (id) => axios.delete(`${BASE}/${id}`)
}

// Utilisation dans App.jsx
const task = await taskService.create({ title: "Nouvelle tache", priority: "High" })
// Axios envoie automatiquement le header Content-Type: application/json
// et parse la reponse JSON en objet JS

// Gestion d'erreur
try {
  await taskService.create(data)
} catch (error) {
  if (error.response?.status === 400) {
    setError("Donnees invalides")
  } else {
    setError("Erreur serveur")
  }
}

```

5.1 Qu'est-ce que REST ?

REST (Representational State Transfer) est une **convention** pour concevoir des APIs HTTP. Une API REST expose des **ressources** (Tasks, Users...) accessibles via des URLs et manipulables avec les verbes HTTP standard.

Verbe HTTP	Action	Exemple URL	Code succes
GET	Lire (liste ou element)	GET /api/tasks	200 OK
POST	Creer	POST /api/tasks	201 Created
PUT	Remplacer/Modifier	PUT /api/tasks/5	200 OK
PATCH	Modifier partiellement	PATCH /api/tasks/5	200 OK
DELETE	Supprimer	DELETE /api/tasks/5	204 No Content

5.2 Format JSON

JSON (JavaScript Object Notation) est le format d'echange universel entre frontend et backend. C'est du texte, lisible par les humains, parse en objet JS ou C# automatiquement.

```
// Corps d'une requete POST /api/tasks
{
  "title": "Apprendre le JSON",
  "description": "Format d'echange entre frontend et backend",
  "priority": "High",
  "dueDate": "2024-03-01T00:00:00Z"
}

// Reponse 201 Created de l'API
{
  "id": 4,
  "title": "Apprendre le JSON",
  "description": "Format d'echange entre frontend et backend",
  "isCompleted": false,
  "priority": "High",
  "createdAt": "2024-01-15T10:30:00Z",
  "dueDate": "2024-03-01T00:00:00Z"
}

// Types JSON : string, number, boolean, null, array [], object {}
// Les dates sont des strings ISO 8601 : "2024-01-15T10:30:00Z"
```

5.3 CORS — Cross-Origin Resource Sharing

Le navigateur bloque les requetes vers un domaine different de la page courante. React sur localhost:5173 ne peut pas appeler localhost:5000 sans permission explicit.

```
// Sans CORS configure, le navigateur affiche :  
// "Access to XMLHttpRequest has been blocked by CORS policy"  
  
// Configuration dans Program.cs :  
builder.Services.AddCors(options =>  
{  
options.AddPolicy("AllowReact", policy =>  
policy  
.WithOrigins("http://localhost:5173") // URL exacte du frontend  
.AllowAnyHeader() // Accepte tous les headers  
.AllowAnyMethod()); // Accepte GET/POST/PUT/DELETE  
});  
  
// Dans le pipeline :  
app.UseCors("AllowReact"); // Avant MapControllers() !
```

6.1 Commandes essentielles

```
# Initialisation
git init # Cree un nouveau depot
git clone URL # Clone un depot distant

# Etat et historique
git status # Fichiers modifies / en attente
git log --oneline # Historique compact
git diff # Differences non stagees

# Cycle de base : modifier -> stager -> committer
git add fichier.cs # Stager un fichier
git add . # Stager tous les fichiers modifies
git commit -m "feat: ajouter X" # Creer un commit

# Branches
git branch feature/auth # Creer une branche
git checkout feature/auth # Basculer sur la branche
git checkout -b feature/auth # Creer + basculer (raccourci)
git merge feature/auth # Fusionner dans la branche courante

# Remote (GitHub)
git remote add origin URL # Lier a GitHub
git push -u origin main # Pousser et tracker la branche
git pull # Recuperer les modifications distantes

# Annuler
git restore fichier.cs # Annuler les modifications non stagees
git reset HEAD fichier.cs # Unstager un fichier
git revert HASH # Creer un commit qui annule un commit
```

6.2 Conventional Commits

Convention de nommage des commits largement adoptee en entreprise. Permet de generer des changelogs automatiques et de rendre l'historique lisible.

```
# Format : <type>(<scope facultatif>): <description>

feat: ajouter le endpoint DELETE /api/tasks
fix: corriger la validation du titre vide
docs: ajouter les commentaires sur ITaskService
refactor: extraire la logique de mapping dans TaskService
test: ajouter les tests unitaires de TaskService
chore: mettre a jour les packages NuGet
style: reformater le CSS des TaskCard
```

```
# Avec scope (composant ou module concerne)
feat(backend): ajouter le filtre par priorite
feat(frontend): ajouter la barre de recherche
fix(auth): corriger le bug de deconnexion

# Breaking change (changement majeur)
feat!: modifier le format de reponse de l'API
BREAKING CHANGE: le champ "done" est renomme "isCompleted"
```

6.3 .gitignore — ce qu'il ne faut pas versionner

```
# C# / .NET : ne pas versionner les binaires et la BDD locale
bin/
obj/
*.db
*.db-shm
*.db-wal

# React / Node : les packages se reinstallent avec npm install
node_modules/
dist/

# Variables d'environnement sensibles (cles API, mots de passe)
.env
.env.local

# OS
.DS_Store # macOS
Thumbs.db # Windows

# IDE
.idea/
.vscode/settings.json
```



```

COPY . .
RUN npm run build # Genere des fichiers statiques dans dist/

FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80

```

7.3 docker-compose.yml

```

version: "3.9"

services:
  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    container_name: todo-backend
    ports:
      - "5000:8080" # host:container
    volumes:
      - db-data:/app/data # Persiste la BDD SQLite
    environment:
      - ASPNETCORE_ENVIRONMENT=Production
    networks:
      - todo-network

  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile
    container_name: todo-frontend
    ports:
      - "3000:80"
    depends_on:
      - backend # Attend que backend soit pret
    networks:
      - todo-network

  volumes:
    db-data: # Volume nomme : persiste entre les redemarrages

  networks:
    todo-network: # Les containers se trouvent par nom ("backend", "frontend")
    driver: bridge

```

7.4 Commandes Docker essentielles

```

# Build et demarrage
docker-compose up --build # Build les images et demarre tout
docker-compose up -d --build # En arriere-plan (detach)
docker-compose down # Arrêter et supprimer les containers

```

```
# Inspecter
docker ps # Containers en cours
docker logs todo-backend # Logs du backend
docker exec -it todo-backend sh # Shell interactif dans le container

# Images et volumes
docker images # Lister les images
docker volume ls # Lister les volumes
docker system prune # Nettoyer les ressources inutilisees

# Build manuel
docker build -t todo-backend ./backend # Construire une image
docker run -p 5000:8080 todo-backend # Lancer un container
```